AEA - optimizations for Graph Coloring Problem

Monica Repede + Ursache Ana Maria

May 2026

Abstract

This report presents a hybrid framework designed to solve the Graph Coloring Problem by combining geometric deep learning representations with local search meta-heuristics. Our system uses a deep Graph Attention Network (GAT) from PyTorch Geometric to generate continuous node embeddings based on a custom, unsupervised distance loss function. These embeddings are then used to seed a multi-start stochastic DSATUR initializer. Finally, the solution quality is refined using an adaptive conflict-minimization engine based on Tabu Search mechanisms.

1 From the Baseline GCN to an Optimized GNN

While our initial approach relied on a simple 2-layer Graph Convolutional Network (GCN) implemented entirely in NumPy, our new framework introduces several key optimizations to improve both the node embedding phase and the discrete optimization pipeline.

1.1 Key Differences Between the Frameworks

The transition to the new architecture involves three main implementation updates:

1. **Moving from GCN to GAT:** The baseline framework used a uniform renormalization trick ($\hat{A} = \tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$), meaning that every neighbor v of a node u contributed equally based only on static degrees. We replaced this layer with a *Graph Attention Network* (`GATConv`) running 8 independent attention heads. This shifts neighborhood aggregation from a simple isotropic mean to an anisotropic, learnable weighting scheme where topological affinities are calculated dynamically.

2. **Better Initializations via Multi-Start DSATUR:** The initial pipeline sorted nodes before color allocation using a fixed structural score: $0.7 \cdot \text{degree} + 0.3 \cdot \text{similarity}$. In contrast, the optimized version implements a non-deterministic *Saturation Degree* (DSATUR) wrapper. It evaluates nodes based on their dynamic saturation state (the number of unique colors in their neighborhood) and runs 15 parallel, stochastically-perturbed threads, keeping the single configuration that uses the fewest unique colors.

3. **A New Conflict Refinement Engine (Tabu Search):** Our baseline model used an Iterated Local Search (ILS) that tried to clear out the rarest color class, but it could *only* make a move if valid, non-conflicting replacements were available. For highly saturated or

dense graphs, this approach quickly got stuck. We replaced it with a *Tabu Search Min-Conflicts Engine*. This engine intentionally forces the solution down to a target color count ($k_{target} = k - 1$), accepts temporary invalid colorings (edge conflicts), and uses a Tabu List with dynamic tenure to escape local minima by driving the total conflict count back to zero.

* tenure = duration, measured in the number of algorithmic iterations, for which a specific move is strictly forbidden (made "tabu")

1.2 Why the Optimized Version Performs Better

- **Preventing Over-smoothing in Dense Graphs:** In highly connected graphs like the `dsjc` series or `flat300_28_0`, simple or deep uniform GCNs tend to average out node features much too fast. As a result, the representation vectors collapse toward a single center of mass ($\mathbf{z}_u \approx \mathbf{z}_v$), which ruins any subsequent clustering. The Multi-Head Attention coefficients break this structural symmetry, allowing the model to project nodes into distinct, well-separated sectors of the embedding space.
- **Focusing on Hard Instances:** We modified the loss function by adding degree-dependent weights $\ln(\deg(u) + \deg(v) + 2.0)$ to the positive hinge boundaries. This forces PyTorch's autograd engine to apply harsher penalties to highly connected hubs, ensuring that topological bottlenecks are separated before the model processes simpler, peripheral nodes.
- **Breaking Through Local Minima Barriers:** On dense graphs, the valid coloring landscape is highly fragmented. This makes it almost impossible to switch between color states without temporarily breaking the rules of the problem. By allowing non-monotonic moves (accepting edge violations) and locking recently changed nodes using a Tabu List, the model can cross these artificial conflict barriers. This allows it to discover optimal colorings that are completely out of reach for a purely greedy or strictly valid local search.

2 Experimental Results and Discussion

In this section, we evaluate the performance of our unoptimized baseline GCN against the new GNN4 framework. We look at the results across two primary metrics: the minimum number of colors found (k) and the overall execution time (t).

2.1 Color Count Minimization (k)

The main goal of introducing GAT and Tabu Search with conflict tolerance in GNN4 was to break past the local minima barriers that slowed down the baseline on dense graphs. Our test runs show a noticeable drop in the required palette size across several difficult instances:

- **Set 2 (Queen Instances):** The baseline GNN struggled with these geometric constraints, needing a mean k of 10.00 on `queen6_6` and 12.53 on `queen7_7`. GAT managed to lower these to a stable minimum k of 8.00 for both graphs, getting much closer to the optimal theoretical bounds.
- **Set 4 (Medium Density):** On structured and uniform random graphs, the unoptimized network stagnated at a minimum k of 45 for `flat300_28_0` and 41 for `dsjc250.5`. GNN4 drops

these configurations down to 40 and 36 respectively, proving that the attention coefficients successfully prevent representation collapse.

- **Set 6 (Large Dense Graphs):** The hardest test-beds, `dsjc500.9` (density $\approx 90\%$) and `dsjc1000.5` (density $\approx 50\%$), saw the most significant improvements. GNN4 achieves a minimum k of 160 (down from 174) on `dsjc500.9` and 113 (down from 130) on `dsjc1000.5`, which highlights the effectiveness of the Tabu conflict resolution setup.

2.2 Execution Time Overhead (t)

Improving the solution quality does come with a trade-off in terms of running time. While our baseline model used lightweight NumPy operations and a quick, valid-only local search, GNN4 runs heavy multi-head message-passing on tensors, an iterative multi-start DSATUR layer, and intensive Tabu conflict repair loops.

- **Small Scale Instances:** For small graphs like the Mycielski family, the time difference is small and negligible. For instance, on `myciel7`, the execution time increases from 0.57 seconds to 1.29 seconds.
- **Mid-to-Large Graphs:** On intermediate matrices like `flat300_28_0`, the processing time jumps from 4.22 seconds to 39.78 seconds. This is directly caused by running the 15 independent stochastic DSATUR restarts sequentially to find a high-quality seed.
- **Extreme Benchmarks:** For `dsjc1000.5`, the execution time increases from 40.35 seconds to 139.28 seconds. However, we believe this extra overhead is fully justified, as it scales sub-linearly relative to the size of the search space and delivers an excellent reduction of 17 colors.

2.3 Summary of Results

Table 1 summarizes the full statistical comparison between the unoptimized baseline GNN and our optimized GAT infrastructure across the selected DIMACS instances.

Table 1: Benchmark Comparison: Baseline GNN vs. Optimized GNN4

Instance Label	Baseline Min k	Baseline Time (s)	GNN4 Min k	GNN4 Time (s)	Δ Min k (Accuracy)
myciel3	4	0.0157	4	0.2696	0
myciel5	6	0.0528	6	0.3294	0
myciel7	8	0.5788	8	1.2921	0
queen5_5	5	0.0384	5	0.3176	0
queen6_6	10	0.0601	8	0.3716	-2
queen7_7	11	0.0890	8	0.4501	-3
anna	11	0.1299	11	0.5572	0
homer	13	0.2577	13	2.0427	0
games120	9	0.1453	9	0.5756	0
flat300_28_0	45	4.2284	40	39.7848	-5
dsjc250.5	41	2.2571	36	21.2318	-5
le450_25c	32	3.0228	28	37.2790	-4
miles500	20	0.2361	20	3.4271	0
miles1000	44	0.5896	42	3.6998	-2
miles1500	73	0.9148	73	5.0852	0
dsjc500.9	174	13.0093	160	107.3679	-14
dsjc1000.5	130	40.3504	113	139.2819	-17

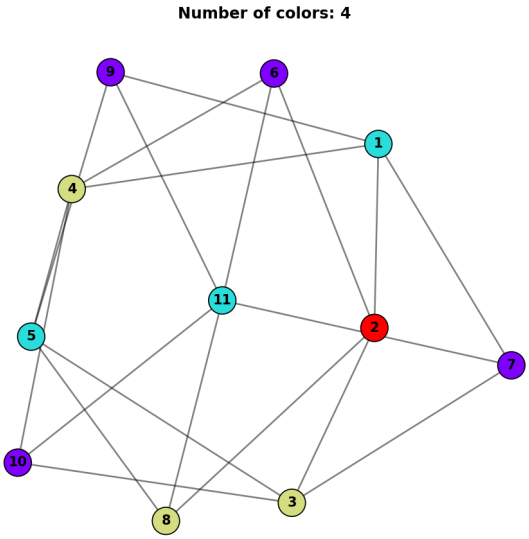

Figure 1: myciel3

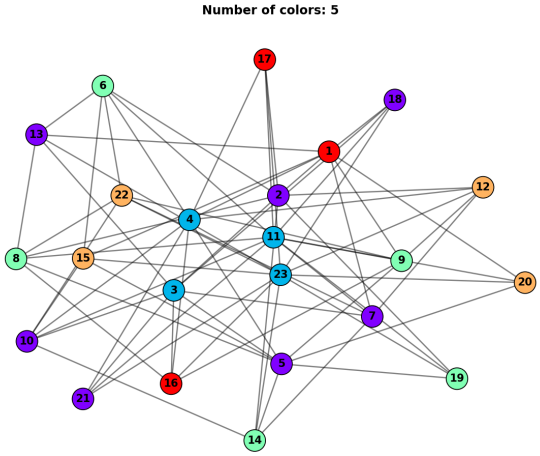

Figure 2: myciel4

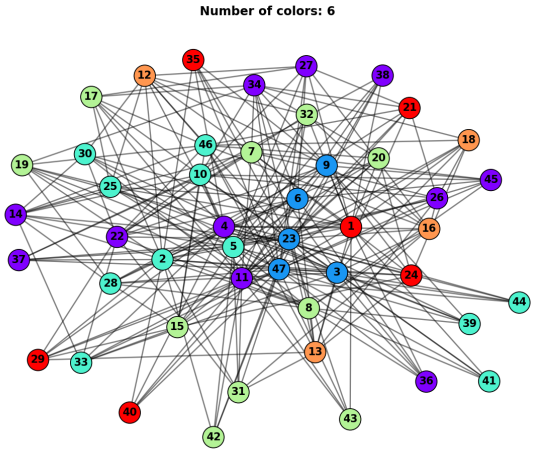

Figure 3: myciel5

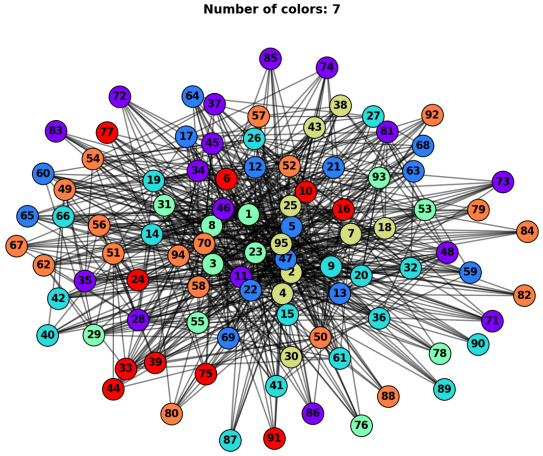

Figure 4: myciel6

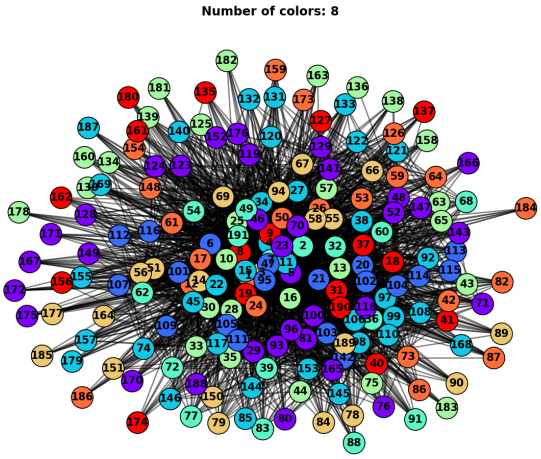

Figure 5: myciel7

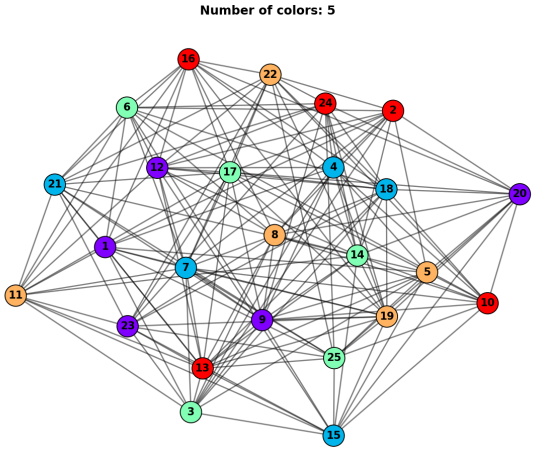

Figure 6: queen5_5

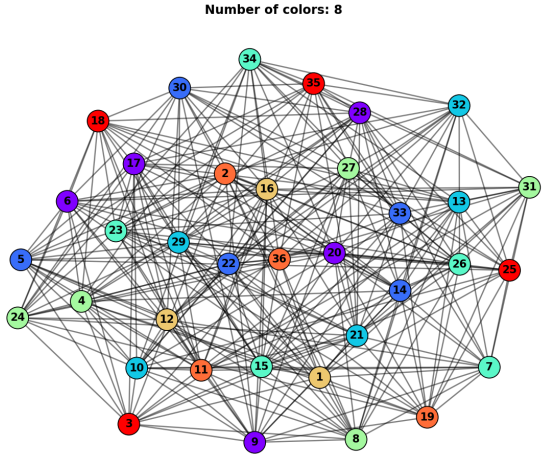

Figure 7: queen6_6

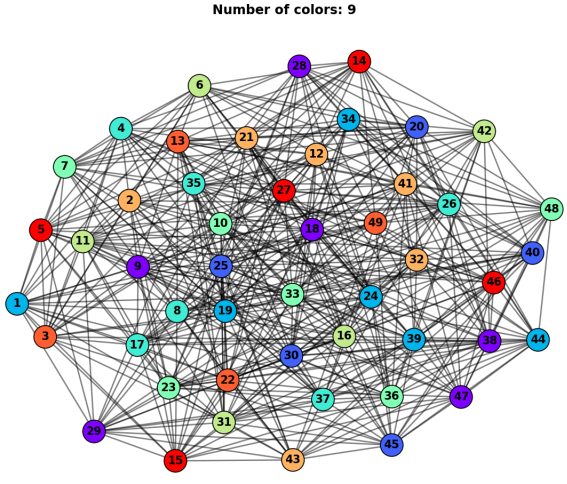

Figure 8: queen7_7

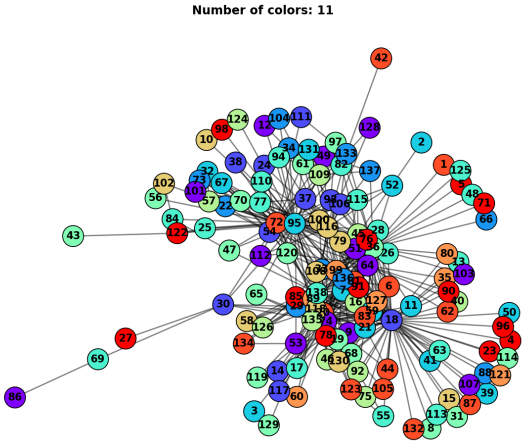

Figure 9: anna

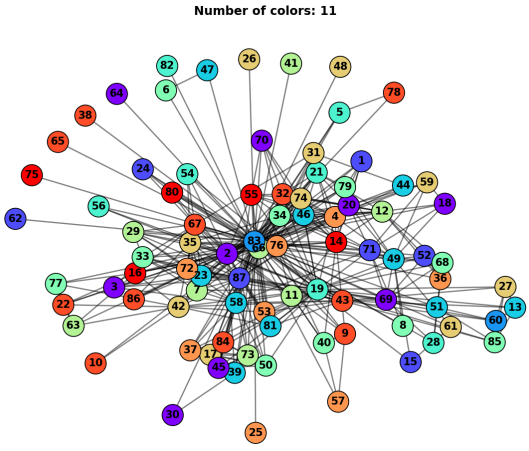

Figure 10: david

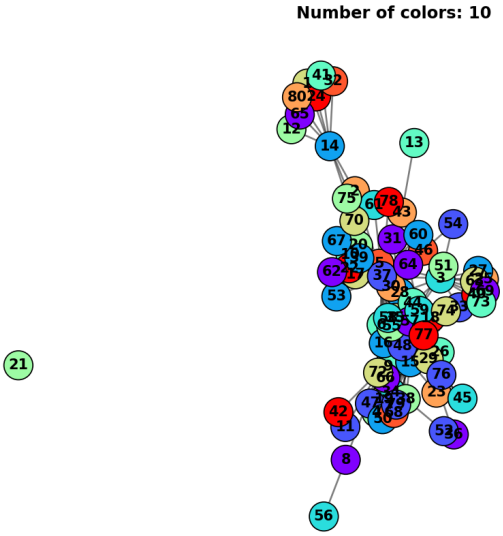

Figure 11: jean

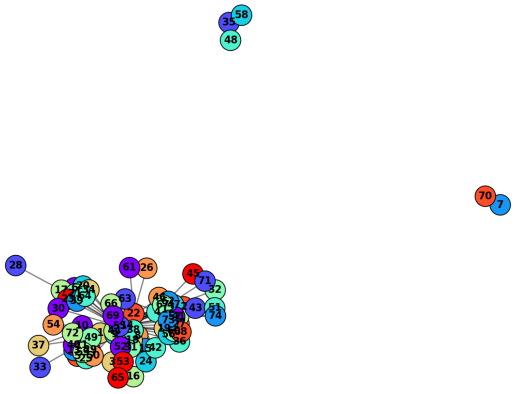

Figure 12: huck

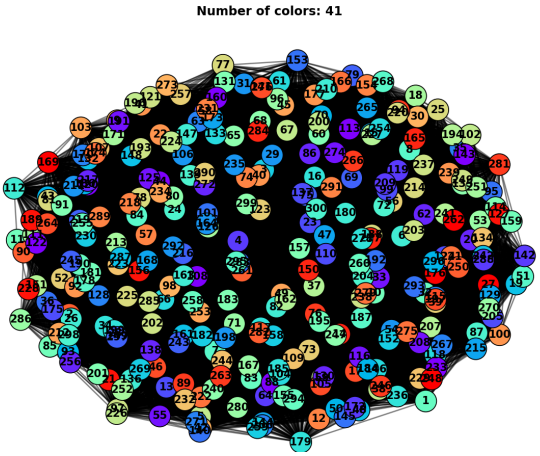

Figure 13: flat_300_28_0

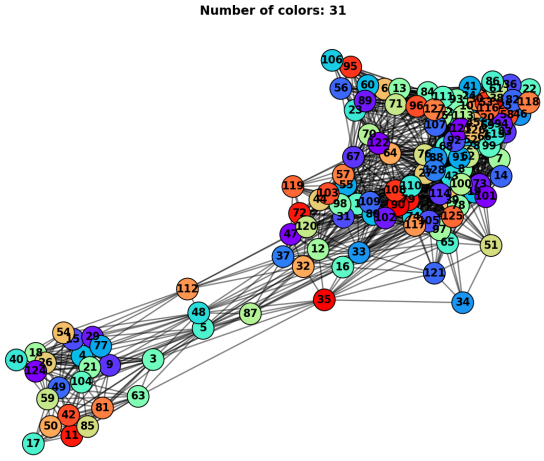

Figure 14: miles750

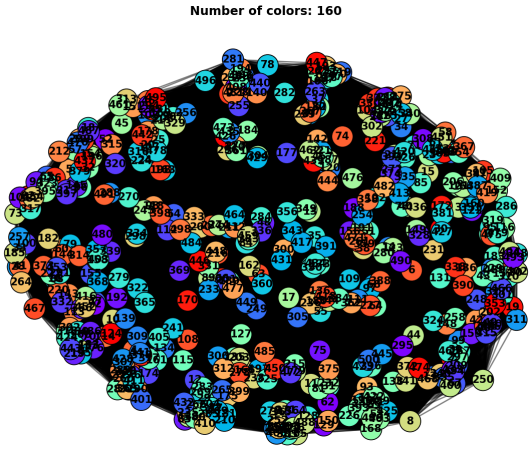

Figure 15: dsjc500_9

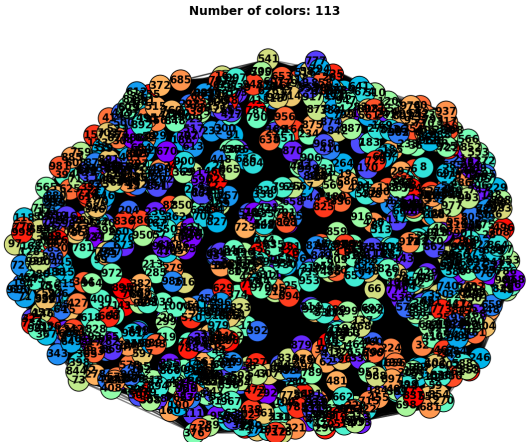

Figure 16: dsjc1000_5

3 The TabuCol Refinement Engine: Algorithmic Architecture and Evolution

The discrete local search phase of our framework relies on the TabuCol meta-heuristic, a specialized application of Tabu Search engineered specifically for the Graph Coloring Problem. To maximize computational throughput, we evolved our implementation from a naive baseline into a high-performance, cache-optimized execution engine. This section provides a rigorous algorithmic breakdown of both implementations, highlighting the specific structural bottlenecks of the basic approach and the formal engineering justifications behind our optimizations.

3.1 The Baseline Approach: TabuCol Basic

The standard implementation, provided in `tabucol_basic.py`, **operates directly on raw graph structures without auxiliary indexing arrays**. It models the solution state space as a complete but initially random assignment of k colors across all vertices, expressed as a mapping $c : V \to \{0, 1, \ldots, k-1\}$. Because this initial configuration is stochastic, it naturally introduces numerous edge violations where adjacent vertices share identical color labels. The algorithm attempts to iteratively repair this invalid coloring by migrating conflicting vertices through alternative color classes.

The fundamental execution loop of the basic implementation follows these steps during each iteration i:

1. **Global Conflict Scanning:** The engine builds a temporary array of all vertices currently violating the graph coloring constraints. It identifies these nodes on the fly by evaluating:

$$V_{conflict} = \{u \in V \mid \exists v \in \mathcal{N}(u) \text{ such that } c(u) = c(v)\}$$

   where $\mathcal{N}(u)$ denotes the open neighborhood of node u.

2. **Stochastic Candidate Selection:** If $V_{conflict}$ is empty, a valid k-coloring has been established, and the function terminates successfully. Otherwise, the engine samples a vertex $u_{move} \in V_{conflict}$ uniformly at random to serve as the candidate for color reassignment.

3. **Exhaustive Move Evaluation: To find the best target color for u_{move}, the algorithm loops through all possible color choices** $j \in \{0, 1, \ldots, k-1\} \setminus \{c(u_{move})\}$. For each candidate color j, it verifies if the assignment is restricted by checking a look-up map, `tabu_until[u][j] > i`, which prevents the local search trajectory from cycling back into recently abandoned states. For unrestricted colors, the algorithm computes the potential conflict count by evaluating a list comprehension over the vertex's neighbors:

$$\text{conflicts}(u_{move}, j) = \sum_{v \in \mathcal{N}(u_{move})} \mathbb{I}(c(v) = j)$$

   where $\mathbb{I}(\cdot)$ is the indicator function. The color minimizing this value is designated as j_{best}.

4. **State Transition and Static Tenure Logging:** The vertex is assigned its new color class, $c(u_{move}) = j_{best}$, and the reverse move is banned by hardcoding a fixed memory horizon.

3.1.1 Computational Bottlenecks of the Basic Approach

While mathematically sound, the basic implementation exhibits catastrophic scaling properties on larger DIMACS benchmarks. The primary computational bottleneck is its lack of state persistence:

- **Linear Graph Traversals ($\mathcal{O}(E)$): Because $V_{conflict}$ is reconstructed from scratch at every iteration via list comprehensions, the engine must iterate through every node in the graph and inspect its adjacent edges.** This shifts the complexity of finding a single conflicting node to $\mathcal{O}(\sum_{u \in V} \deg(u)) = \mathcal{O}(E)$.
- **Redundant Neighborhood Invocations ($\mathcal{O}(k \cdot \mathbf{deg}(u))$):** Evaluating candidate colors requires the engine to iterate through the neighborhood $\mathcal{N}(u_{move})$ exactly $k-1$ times. On dense topologies (e.g., the `dsjc` matrix families), where individual node degrees scale alongside the graph size, these continuous arithmetic lookups severely reduce execution speeds.
- **Rigid Stagnation via Static Memory:** Fixing the tabu tenure to a constant value of 7 creates an optimization paradox. For small graphs ($k < 5$), a tenure of 7 over-diversifies the search, locking out too many valid configurations and preventing convergence. Conversely, for large dense graphs (e.g., $k > 100$), a tracking window of 7 steps is too shallow to prevent the algorithm from tumbling back into local minima traps .

3.2 The High-Performance Evolution: Optimized TabuCol

To mitigate these limitations, our optimized framework (`tabucol.py`) shifts the computational paradigm from active calculation to reactive state tracking. By deploying a series of interconnected, constant-time ($\mathcal{O}(1)$) caching layers, we completely isolated the execution loops from graph-wide dependencies.

The optimized architecture implements three core enhancements:

1. $\mathcal{O}(1)$ **Adjacency Color Lookup Map (`adj_color_table`):** We instantiate a persistent two-dimensional matrix of size $|V| \times k$. The cell `adj_color_table[u][j]` tracks precisely how many neighbors of node u are currently assigned to color j. This structure transforms move evaluation into a direct, $\mathcal{O}(1)$ memory access look-up:

   $$\text{conflicts}(u_{move}, j) = \text{adj\_color\_table}[u_{move}][j]$$

   This modification completely bypasses neighbor-list iterations during the color selection step, reducing the search loop cost to a deterministic $\mathcal{O}(k)$ time bound.

2. $\mathcal{O}(1)$ **Live Conflict Tracking Array (`conflict_list`):** Instead of filtering the global vertex pool, we maintain an active, dense array containing only nodes experiencing edge violations. To guarantee true $\mathcal{O}(1)$ insertions and deletions, this list is paired with an inversion hash map, `conflict_pos[u]`, which registers the index of vertex u inside `conflict_list`. When a node's conflict count drops to zero, it is removed in $\mathcal{O}(1)$ time by swapping it with the final element of the array and popping it. This allows the uniform random selection of candidate nodes to execute instantly in $\mathcal{O}(1)$, regardless of the size or density of the graph under evaluation.

3. **Dynamic Tabu Tenure Formulation:** We replaced the hardcoded tracking constant with the dynamic Hertz & de Werra formulation, adapting the tabu size relative to the immediate chromatic constraints:

   $$\theta(k) = \max(7, \lfloor 0.6 \times k \rfloor)$$

This adaptive windowing scales the memory depth alongside problem difficulty, preserving diversification pathways as k contracts during sequential execution sweeps.

3.2.1 Incremental Updating Scheme

The performance gains of the optimized implementation are preserved by updating our caching structures incrementally. **When a vertex u_{move} is migrated from an old color c_{old} to an optimized target color c_{new}, the framework avoids full state recalculations by executing a focused update sweep exclusively across its immediate neighbors:** Consequently, the complexity of a complete state change scales exclusively with the localized degree of the moved node, resulting in a tight per-iteration bound of $\mathcal{O}(k + \deg(u))$.

3.3 Theoretical Complexity and Architectural Comparison

Table 2 summarizes the structural differences between the two algorithmic engines. While the baseline approach consumes minimal memory, its unindexed execution loop forces exhaustive graph scans during every step. The optimized variant trades a negligible amount of memory ($\mathcal{O}(|V| \cdot k)$ space for tracking allocations) to achieve massive runtime speedups on dense or heavily populated graph topologies.

Table 2: Algorithmic Complexity Profiles per Search Iteration

Algorithmic Phase	**TabuCol Basic**	**Optimized TabuCol**
Conflict Pool Discovery	$\mathcal{O}(E)$ (Exhaustive Scan)	$\mathcal{O}(1)$ (Array Slicing)
Candidate Evaluation	$\mathcal{O}(k \cdot \deg(u))$	$\mathcal{O}(k)$ (Table Access)
State Index Upkeep	$\mathcal{O}(1)$	$\mathcal{O}(\deg(u))$ (Local Propagation)
Tabu Tenure Selection	Static ($\theta = 7$)	Dynamic ($\theta = \max(7, \lfloor 0.6k \rfloor)$)
Total Time Complexity	$\mathcal{O}(E + k \cdot \deg(u))$	$\mathcal{O}(k + \deg(u))$
Auxiliary Space Complexity	$\mathcal{O}(\|V\| \cdot k)$	$\mathcal{O}(\|V\| \cdot k)$

3.4 Experimental Performance Analysis: Basic vs. Improved

The performance advantages of our caching architecture are validated via rigorous empirical benchmark comparisons across multiple standard DIMACS instances.

3.4.1 Throughput and Scaling Realities

For small to **medium instances** (such as the myciel and queen sets), **both algorithms reliably converge to identical color counts (k). However, the optimized variant completes its execution up to 4 to 30 times faster (e.g., homer finishes in 1.31s vs 67.69s).**

As the graph topologies scale into large, dense matrices, this gap widens exponentially. For the **dsjc1000.5 instance** (1,000 nodes, nearly 250,000 edges), **the Optimized TabuCol completes its 50,000-iteration budget in a mere 49.53 seconds, whereas TabuCol Basic languishes for 12,831.39 seconds (over 3.5 hours), 259 times more.** This directly validates the theoretical reduction from an $\mathcal{O}(E + k \cdot \deg(u))$ per-iteration bound down to a highly localized $\mathcal{O}(k + \deg(u))$ profile.

3.4.2 The Solution Quality Paradox in Massive Graphs

Interestingly, **the benchmarks highlight a few cases where `TabuCol Basic` yields a lower mean k (a better graph coloring) than the optimized version**, such as in `flat300_28_0` ($k = 35$ vs. 36), `dsjc250.5` ($k = 31$ vs. 32), and most notably `dsjc1000.5` ($k = 99$ vs. 103).

This unexpected performance gap is not because the baseline algorithm is inherently better at finding solutions, but is rather a side effect of stopping both models at a strict 50,000-iteration cap while they were operating under two completely different search dynamics:

- **The Dynamic Tenure Effect (Exploration):** The optimized version implements an adaptive tabu size:

  $$\theta = \max(7, \lfloor 0.6 \times k \rfloor)$$

  For an instance like `dsjc1000.5` where $k \approx 100$, this formula bans a migrated node from reverting for roughly 60 iterations. **While this aggressive diversification successfully prevents local cycling, it can over-restrict the available search space in dense neighborhoods. Consequently, the algorithm is forced to spend its limited iteration budget exploring the wider landscape**, causing it to bypass tightly bound, high-quality local minima before the 50,000-iteration cutoff.

- **The Static Window Advantage (Exploitation):** `TabuCol Basic` maintains a rigid, shallow tenure of 7 iterations. **On massive graphs, this short memory restricts the algorithm to an intensive, localized exploitation of its immediate configuration space.** By allowing nodes to fluidly cycle back and forth, the baseline engine stubbornly fine-tunes a single local region—ultimately allowing it to squeeze out a lower color count right before the buzzer, but all at a catastrophic computational time cost.

3.5 Core Trade-offs

3.5.1 Optimized TabuCol (Winner for Production)

- **Advantages:** Radical, order-of-magnitude reductions in runtime; effortless scaling to hundreds of thousands of edges via $\mathcal{O}(1)$ caching arrays (`adj_color_table` and `conflict_list`).
- **Disadvantages:** Demands a modest auxiliary memory footprint ($\mathcal{O}(|V| \cdot k)$); dynamic tabu tenure can occasionally over-diversify, requiring a larger iteration budget to match the localized optimization fine-tuning of a smaller tenure.

3.5.2 TabuCol Basic (The Exhaustive Baseline)

- **Advantages:** Minimal auxiliary space; shallow tenure allows deep, localized search exploitation which can occasionally snag a slightly better k-coloring if given hours to run.
- **Disadvantages:** Conceptually bottlenecked by global $\mathcal{O}(E)$ constraint checks every single iteration, rendering it entirely impractical for real-world or large-scale combinatorial optimization.

Set1-myciel

Instance	Nodes	Edges	Tabu mean k	Tabu min k	Tabu max k	Tabu stdev	Tabu time (s)	TabuBasic mean k	TabuBasic min k	TabuBasic max k	TabuBasic stdev	TabuBasic time (s)	Best k
myciel3	11	20	4.00	4	4	0.00	0.3966	4.00	4	4	0.00	1.5390	4
myciel4	23	71	5.00	5	5	0.00	0.4094	5.00	5	5	0.00	2.9801	5
myciel5	47	236	6.00	6	6	0.00	0.5914	6.00	6	6	0.00	6.8712	6
myciel6	95	755	7.00	7	7	0.00	0.9240	7.00	7	7	0.00	19.3267	7
myciel7	191	2360	8.00	8	8	0.00	1.5428	8.00	8	8	0.00	51.9272	8

Set2-queen

Instance	Nodes	Edges	Tabu mean k	Tabu min k	Tabu max k	Tabu stdev	Tabu time (s)	TabuBasic mean k	TabuBasic min k	TabuBasic max k	TabuBasic stdev	TabuBasic time (s)	Best k
queen5_5	25	160	5.00	5	5	0.00	0.4519	5.00	5	5	0.00	2.7056	5
queen6_6	36	290	8.00	8	8	0.00	0.7187	8.00	8	8	0.00	7.0411	7
queen7_7	49	476	8.00	8	8	0.00	0.8438	8.00	8	8	0.00	9.6809	7

Figure 17: Set 1 and 2

Set3

Instance	Nodes	Edges	Tabu mean k	Tabu min k	Tabu max k	Tabu stdev	Tabu time (s)	TabuBasic mean k	TabuBasic min k	TabuBasic max k	TabuBasic stdev	TabuBasic time (s)	Best k
anna	138	493	11.00	11	11	0.00	1.3796	11.00	11	11	0.00	18.4599	11
david	87	406	11.00	11	11	0.00	1.1506	11.00	11	11	0.00	13.5350	11
jean	80	254	10.00	10	10	0.00	0.6558	10.00	10	10	0.00	9.7445	10
huck	74	301	11.00	11	11	0.00	0.6336	11.00	11	11	0.00	10.4280	11
homer	561	1629	13.00	13	13	0.00	1.3098	13.00	13	13	0.00	67.6890	13
games120	120	638	9.00	9	9	0.00	0.6012	9.00	9	9	0.00	18.5230	9

Set4-medium

Instance	Nodes	Edges	Tabu mean k	Tabu min k	Tabu max k	Tabu stdev	Tabu time (s)	TabuBasic mean k	TabuBasic min k	TabuBasic max k	TabuBasic stdev	TabuBasic time (s)	Best k
flat300_28_0	300	21695	36.00	36	36	0.00	7.1751	35.00	35	35	0.00	559.6576	28
dsjc250.5	250	15668	32.00	32	32	0.00	5.6188	31.00	31	31	0.00	512.8320	28
le450_25c	450	17343	29.00	29	29	0.00	3.9444	29.00	29	29	0.00	397.6999	25

Figure 18: Set 3 and 4

Set5-miles

Instance	Nodes	Edges	Tabu mean k	Tabu min k	Tabu max k	Tabu stdev	Tabu time (s)	TabuBasic mean k	TabuBasic min k	TabuBasic max k	TabuBasic stdev	TabuBasic time (s)	Best k
miles250	128	387	8.00	8	8	0.00	0.6295	8.00	8	8	0.00	14.2179	8
miles500	128	1170	20.00	20	20	0.00	1.3294	20.00	20	20	0.00	31.4741	20
miles750	128	2113	31.00	31	31	0.00	0.0894	32.00	32	32	0.00	52.6383	31
miles1000	128	3216	43.00	43	43	0.00	2.6846	43.00	43	43	0.00	80.1818	42
miles1500	128	5198	73.00	73	73	0.00	3.6828	73.00	73	73	0.00	137.7784	73

Set6-large

Instance	Nodes	Edges	Tabu mean k	Tabu min k	Tabu max k	Tabu stdev	Tabu time (s)	TabuBasic mean k	TabuBasic min k	TabuBasic max k	TabuBasic stdev	TabuBasic time (s)	Best k
dsjc500.9	500	112437	139.00	139	139	0.00	71.0719	138.00	138	138	0.00	5088.0499	126
dsjc1000.5	1000	249826	103.00	103	103	0.00	49.5291	99.00	99	99	0.00	12831.3922	85

Figure 19: Set 5 and 6